Seminar 2 — Solutions

Advanced Software Security · Network reconnaissance & traffic analysis

These are the solutions for all 10 exercises in Seminar 2.

1 List hosts that respond to ping

```
nmap -sn -n -oG - 10.200.99.0/24 |
  grep 'Status: Up' |
  cut -d ' ' -f 2 > live-hosts.lst
```

CAPTURED RESULT — LIVE-HOSTS.LST (RUN ON THE LOCAL 172.16.98.0/24 SUBNET)

```
172.16.98.71
172.16.98.103
```

2 Make `wget` print `Welcome!` on every request

In one terminal:

```
while true; do
  printf 'HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nContent-Length:
8\r\nConnection: close\r\n\r\nWelcome!' |
    nc -l -p 1234
done
```

In another terminal:

```
wget -qO- http://localhost:1234/
```

3 Transfer `wget` between two `nc` instances

On the receiving VM:

```
nc -l 9000 > wget.copy
```

On the sending VM, replace `<RECEIVER_IP>` with the receiver's lab IP:

```
nc -N <RECEIVER_IP> 9000 < "$(command -v wget)"
```

Then check the copy on the receiver:

```
cmp "$(command -v wget)" wget.copy
chmod +x wget.copy
./wget.copy --version
```

4 Capture DHCP packets and a TCP handshake in Wireshark

We start a capture on the VM's active interface. For DHCP, we use the display filter `dhcp` (older Wireshark versions may use `bootp`). In the isolated lab, we trigger a lease exchange, for example:

```
sudo ipconfig set en0 DHCP
```

We capture the DHCP exchange and take the screenshot. Releasing a lease can briefly interrupt that VM's network.

For the handshake, we generate a TCP connection to a reachable service, then use `tcp.flags.syn == 1` to locate the opening SYN. Note that this filter only matches the SYN and SYN/ACK packets — the final, pure ACK has no SYN flag, so it is not shown by this filter alone. To see the complete three-way handshake, we right-click the SYN packet and choose **Follow → TCP Stream** (equivalently, filter `tcp.stream eq <n>`): the stream then shows SYN, SYN/ACK, and ACK together.

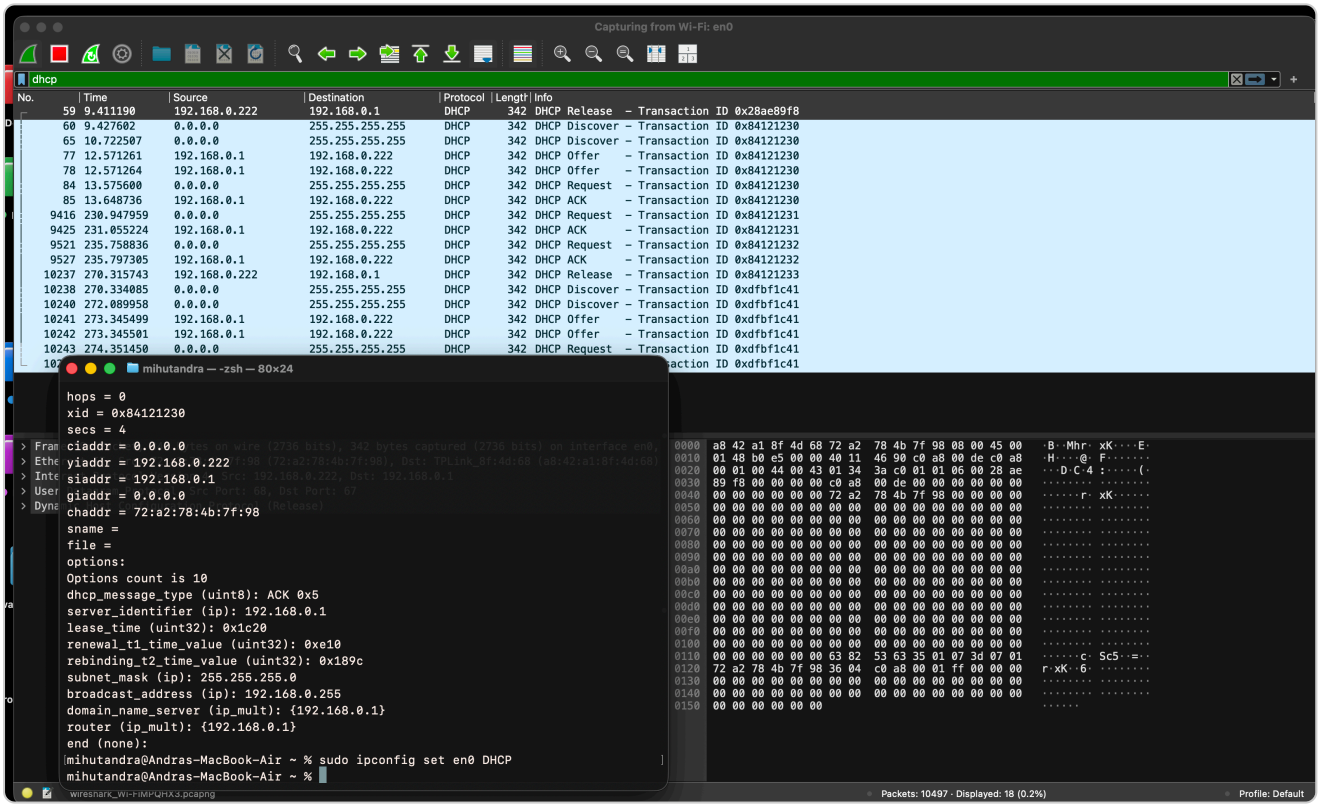


Figure 4a — DHCP exchange (Discover / Offer / Request / ACK, plus Release) with the dhcp display filter.

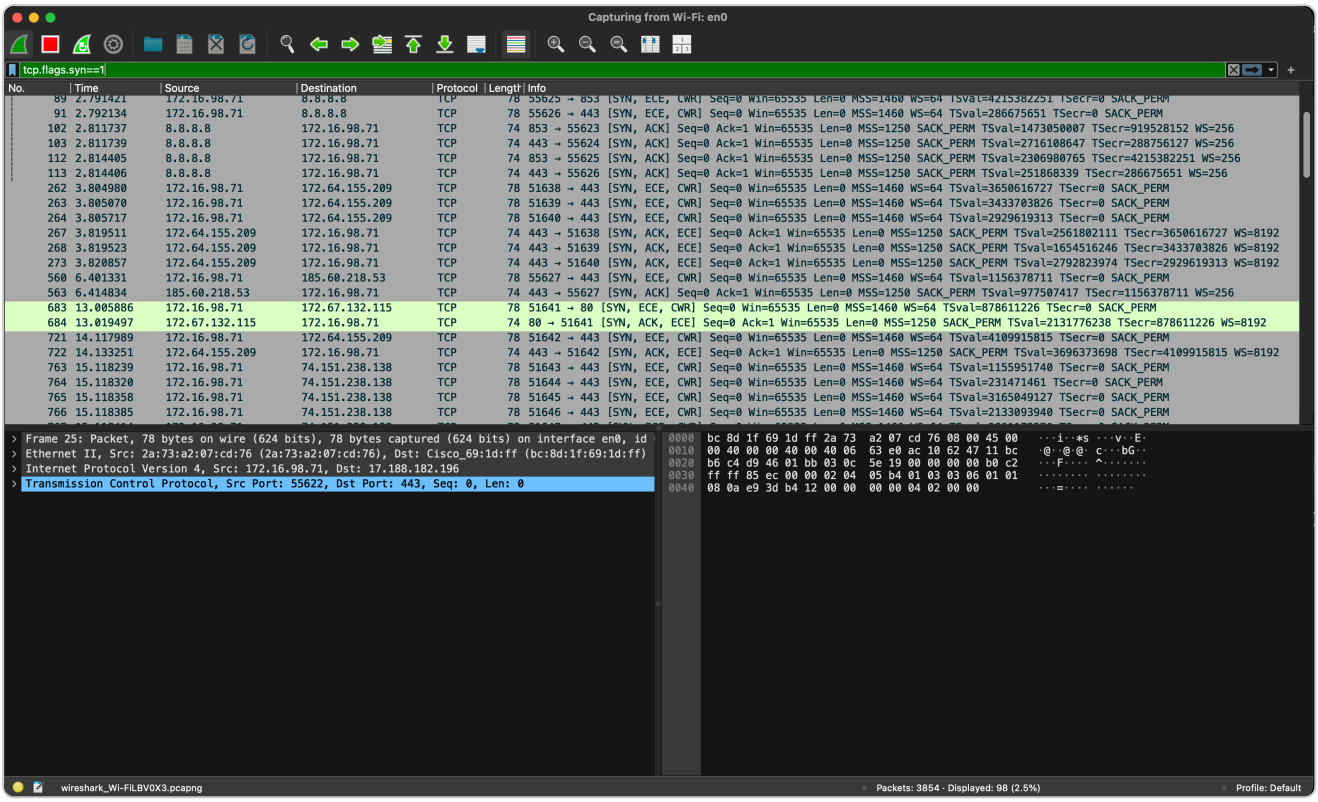


Figure 4b — TCP three-way handshake located with tcp.flags.syn==1 (SYN / SYN-ACK highlighted; full SYN/SYN-ACK/ACK shown via Follow TCP Stream).

5 Show cookies in HTTP GET requests; handle HTTPS

Replace `<CLIENT_IP>` with your VM's IP:

```
sudo tshark -i <IFACE> \
  -Y 'ip.src == <CLIENT_IP> && http.request.method == "GET" && http.cookie' \
  -T fields -e http.request.full_uri -e http.cookie
```

Plain HTTP exposes cookies to packet capture. HTTPS encrypts them; to inspect HTTPS, you need to decrypt the session using authorized TLS session keys (for example, a browser key log configured in Wireshark) or capture through a trusted debugging proxy. Without decryption, the cookie header is not visible.

CAPTURED RESULT — EX5-COOKIES.TXT (PLAIN-HTTP GETS; COOKIES VISIBLE ON THE WIRE)

```
http://httpforever.com/ session=abc123; user=andras
http://neverssl.com/    session=abc123; user=andras
```

6 Explain the HTTP POST filter

```
tcp dst port 80 and (tcp[((tcp[12:1] & 0xf0) >> 2):4] = 0x504f5354)
```

It selects TCP packets going to port 80. The high four bits of TCP byte 12 give the TCP header length in 32-bit words; shifting right by two converts that to bytes. The filter reads four bytes immediately after the TCP header—the start of the TCP payload—and compares them with `0x504f5354`, ASCII `POST`. This matches packets where the payload begins with `POST`, assuming the request line is in that packet.

7 Create, list, and remove `test1` over FTP

Using `nc`: connect to the control port:

```
nc <TARGET> 21
```

We enter these FTP commands, waiting for each server reply:

```
USER service
PASS service
MKD test1
PASV
```

The server's `227` reply gives a passive data address and port as six numbers: `h1,h2,h3,h4,p1,p2`. The data port is `p1*256+p2`. In a second terminal, we connect to that port with `nc <TARGET> <DATA_PORT>`, then send `LIST` in the original control connection. The listing appears in the data connection. Finally, in the control connection:

```
RMD test1
QUIT
```

Using the `ftp` client:

```
ftp <TARGET> 21
```

We log in as `service` with password `service`, then enter:

```
mkdir test1
ls
rmdir test1
quit
```

8 List open TCP ports and detected service versions

In the lab, we scan all TCP ports and save the report:

```
nmap -sV -p- -T4 <TARGET> -oN scan.txt
```

We extract only open ports, service names, and version text:

```
awk '/^[0-9]+\/tcp[[:space:]]+open/ {
  port=$1; service=$3
  $1=$2=$3=""
  sub(/^[[:space:]]+/, "")
  print port, service, $0
}' scan.txt
```

CAPTURED RESULT — EX8-OPEN-PORTS.TXT (LOCALHOST SCAN)

```
5000/tcp rtsp
7000/tcp rtsp
8021/tcp tcpwrapped
46498/tcp unknown
55452/tcp unknown
55456/tcp unknown
55465/tcp unknown
55784/tcp http Node.js Express framework
55787/tcp http Node.js Express framework
63252/tcp unknown
63442/tcp http Node.js Express framework
63781/tcp bandwidth-test MikroTik bandwidth-test server
63796/tcp unknown
63849/tcp unknown
```

9 Compare TCP and UDP scans

We capture traffic on the interface that reaches the target:

```
sudo tcpdump -ni <IFACE> 'host <TARGET> and (tcp or udp or icmp)'
```

In another terminal, we run the scans:

```
sudo nmap -sS -p 21,53,80,1999 <TARGET>
sudo nmap -sU -p 21,53,80,1999 <TARGET>
```

A TCP SYN scan sends SYN probes. Open ports generally reply SYN/ACK; closed ports reply RST. Nmap's SYN scan does not complete the TCP handshake. UDP has no handshake: a closed port often replies with ICMP port-unreachable, while an open service may reply at the application level or remain silent. Silence can leave a port classified as `open|filtered`.

Blocking ICMP can hide UDP "closed" replies and make UDP results less certain. It can also interfere with Nmap's host discovery, so `-Pn` tells Nmap to skip that discovery step. TCP port responses such as SYN/ACK and RST still provide information unless the firewall blocks those too.

CAPTURED RESULT — EX9-CAPTURE.TXT (TCPDUMP ON LO0 DURING THE SCANS)

```
12:10:53.781850 IP 127.0.0.1.61811 > 127.0.0.1.21: Flags [S], seq 1885007448, win 1024, options
[mss 1460], length 0
12:10:53.781861 IP 127.0.0.1.61811 > 127.0.0.1.80: Flags [S], seq 1885007448, win 1024, options
[mss 1460], length 0
12:10:53.781867 IP 127.0.0.1.61811 > 127.0.0.1.53: Flags [S], seq 1885007448, win 1024, options
[mss 1460], length 0
12:10:53.781872 IP 127.0.0.1.61811 > 127.0.0.1.7000: Flags [S], seq 1885007448, win 1024, options
[mss 1460], length 0
12:10:53.781878 IP 127.0.0.1.61811 > 127.0.0.1.1999: Flags [S], seq 1885007448, win 1024, options
[mss 1460], length 0
12:10:53.781882 IP 127.0.0.1.61811 > 127.0.0.1.5000: Flags [S], seq 1885007448, win 1024, options
[mss 1460], length 0
12:10:53.781905 IP 127.0.0.1.21 > 127.0.0.1.61811: Flags [R.], seq 0, ack 1885007449, win 0,
length 0
12:10:53.781920 IP 127.0.0.1.80 > 127.0.0.1.61811: Flags [R.], seq 0, ack 1885007449, win 0,
length 0
12:10:53.781935 IP 127.0.0.1.53 > 127.0.0.1.61811: Flags [R.], seq 0, ack 1885007449, win 0,
length 0
12:10:53.782026 IP 127.0.0.1.7000 > 127.0.0.1.61811: Flags [S.], seq 4263219700, ack 1885007449,
win 65535, options [mss 16344], length 0
12:10:53.782043 IP 127.0.0.1.1999 > 127.0.0.1.61811: Flags [R.], seq 0, ack 1885007449, win 0,
length 0
12:10:53.782086 IP 127.0.0.1.5000 > 127.0.0.1.61811: Flags [S.], seq 2380124398, ack 1885007449,
win 65535, options [mss 16344], length 0
12:10:53.782099 IP 127.0.0.1.61811 > 127.0.0.1.7000: Flags [R], seq 1885007449, win 0, length 0
12:10:53.782104 IP 127.0.0.1.61811 > 127.0.0.1.5000: Flags [R], seq 1885007449, win 0, length 0
12:10:53.905420 IP 127.0.0.1.49298 > 127.0.0.1.1999: UDP, length 0
12:10:53.905432 IP 127.0.0.1.49298 > 127.0.0.1.21: UDP, length 0
12:10:53.905440 IP 127.0.0.1.49298 > 127.0.0.1.80: UDP, length 14
12:10:53.905444 IP 127.0.0.1.49298 > 127.0.0.1.53: 6+ TXT CHAOS? version.bind. (30)
12:10:53.905452 IP 127.0.0.1.49298 > 127.0.0.1.53: 0 stat [0q] (12)
12:10:53.905463 IP 127.0.0.1.49298 > 127.0.0.1.53: 0 PTR? _services._dns-sd._udp.local. (46)
12:10:53.905493 IP 127.0.0.1 > 127.0.0.1: ICMP 127.0.0.1 udp port 1999 unreachable, length 36
12:10:53.905521 IP 127.0.0.1 > 127.0.0.1: ICMP 127.0.0.1 udp port 21 unreachable, length 36
12:10:53.905529 IP 127.0.0.1 > 127.0.0.1: ICMP 127.0.0.1 udp port 80 unreachable, length 36
12:10:53.905542 IP 127.0.0.1 > 127.0.0.1: ICMP 127.0.0.1 udp port 53 unreachable, length 36
```

10 PHP and JavaScript shell scripts

To check connectivity without providing a shell, we use a fixed health-check request instead. For example, browser JavaScript can request an HTTPS endpoint:

```
fetch("https://10.0.0.20/health")
  .then(response => console.log(response.status))
  .catch(error => console.error(error));
```